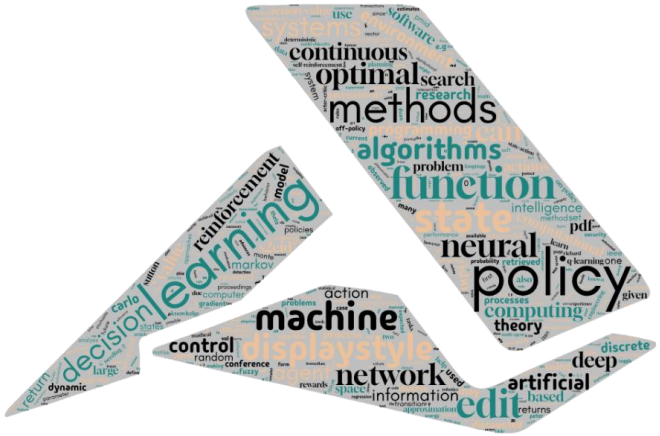


Reinforcement Learning Algorithms and Applications

Chapter 5: State-Action-Reward-State-Action (SARSA)

Introduction to Temporal Difference Learning

- **Temporal Difference (TD)**
  - A method that **updates value estimates** using the **current estimate** and the **estimate of the next step**.
  - The key idea of TD learning is that it **does not need to wait until the end of an episode**.
  - Instead, it updates the current value estimate **at every step** using the estimated value of the next state.
- Both **SARSA** and **Q-Learning** are *on-policy* and *off-policy* TD control methods respectively.
- These algorithms learn directly from experience without requiring a *model* of the environment.
- The key difference lies in how they update their **Q-values** during the learning process.
- Both methods use the *epsilon-greedy* strategy for action selection.
- The **Gridworld environment** provides a perfect testbed for comparing these algorithms.
- Understanding their differences is crucial for selecting the right algorithm for specific tasks.

Q-Learning Algorithm Overview

- **Q-Learning** is an *off-policy* temporal difference control algorithm.
- It learns the optimal action-value function regardless of the policy being followed.
- It is an **off-policy** method.
- **Off-policy**: The agent learns a *target strategy* that is **different** from the strategy it actually uses to take actions.
- The algorithm updates Q-values using the *maximum* Q-value of the next state.
- **Q-Learning update rule**: $Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$
- Uses $\max_{a'} Q(s',a')$ instead of the actual next action's Q-value.
- This makes Q-Learning more **aggressive** in learning the optimal policy.
- Guaranteed to converge to the optimal policy under certain conditions.

SARSA Algorithm Overview

- **SARSA** stands for *State-Action-Reward-State-Action*, representing the tuple used for updates.
- It is an **on-policy** method that learns the value of the policy being followed.
- **On-policy**: The agent learns from the *same strategy* it actually uses to take actions.
- The algorithm updates Q-values using the *actual next action* chosen by the current policy.
- **SARSA update rule**: $Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma Q(s',a') - Q(s,a)]$
- The *next action* a' is selected using the same epsilon-greedy policy.
- This makes SARSA more **conservative** as it considers the exploration in its updates.
- The algorithm converges to the optimal policy under certain conditions.

Key Implementation Differences

- **SARSA** requires selecting the *next action* before updating Q-values.
- **Q-Learning** only needs the *maximum Q-value* of the next state for updates.
- In SARSA: `next_action = agent.get_action(next_state)` is called before update.
- In Q-Learning: `np.max(self.q_table[next_state])` is used directly in update.
- **SARSA update method** takes next_action as a parameter.
- **Q-Learning update method** does not need the next action parameter.
- Both algorithms share the same *epsilon-greedy* action selection strategy.

Training Loop Differences

- **SARSA training loop** selects the first action before entering the step loop.
- **Q-Learning** selects actions within each step of the loop.
- SARSA: action = agent.get_action(state) called at episode start.
- Q-Learning: action selection happens inside the step loop only.
- **SARSA** maintains the *action* variable throughout the episode.
- **Q-Learning** treats each step independently for action selection.
- Both methods use the same *episode* and *step* structure for training.

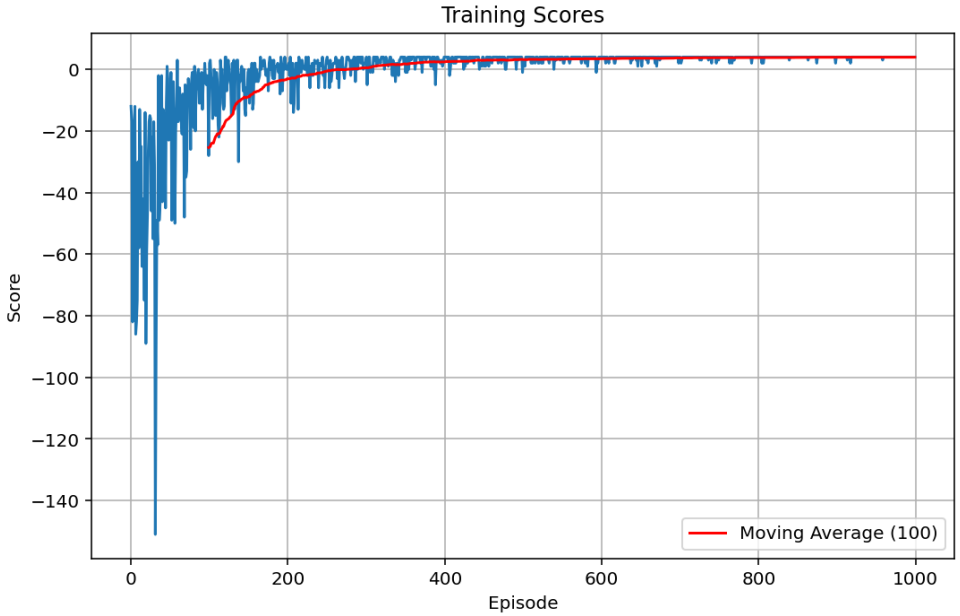
Performance Characteristics

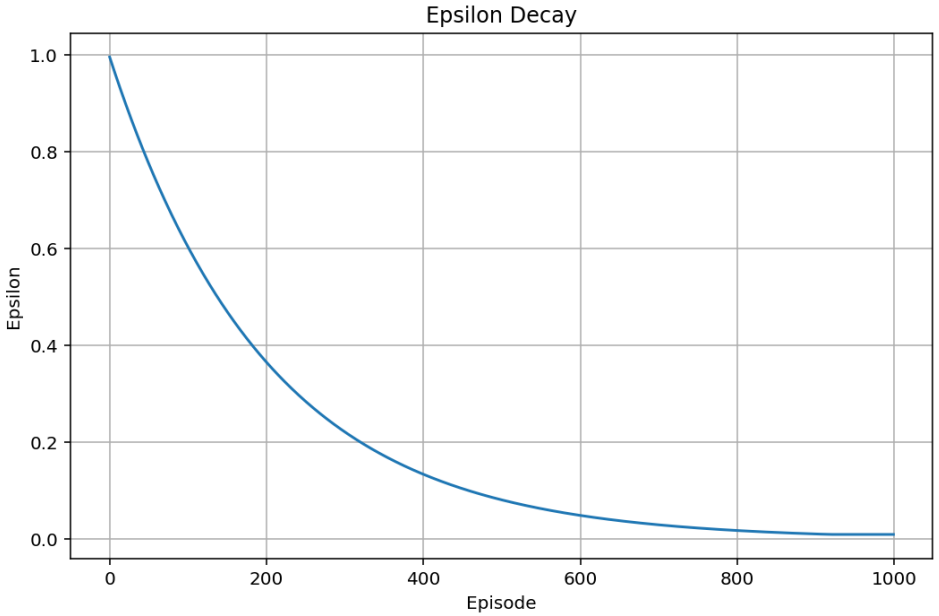
- **SARSA** tends to be more *cautious* near dangerous states like pits.
- **Q-Learning** is more *aggressive* and finds optimal paths faster.
- SARSA considers the *exploration noise* in its value function estimation.
- Q-Learning learns the *optimal Q-function* regardless of current policy.
- Both algorithms achieve similar **final performance** in deterministic environments.
- **Convergence speed** may differ depending on the exploration strategy.
- The choice between them depends on whether *safety* or *optimality* is prioritized.

Practical Implementation Results

- Both algorithms successfully learn to navigate the **4×4 Gridworld**.
- **Training scores** and *epsilon decay* curves show similar patterns.
- Final **Q-tables** may differ slightly due to different update mechanisms.
- Both methods achieve **positive average scores** after sufficient training.
- **Policy visualization** shows both algorithms learn effective navigation strategies.
- *Test performance* demonstrates successful learning in both cases.
- The **demonstrate_learned_policy** function shows the learned behavior clearly.

Practical Implementation Results





Reference

- M. Lapan, *Deep Reinforcement Learning Hands-On*, 2nd ed. Birmingham, UK: Packt Publishing, 2020.
- A. Zai and B. Brown, *Deep Reinforcement Learning in Action*. Shelter Island, NY: Manning Publications, 2020.
- S. Ravichandiran, *Deep Reinforcement Learning with Python*, 2nd ed. Birmingham, UK: Packt Publishing, 2020.